Why are LLM-driven Agentic Data Analysis Slow?

Shannon Kang, Hengrui Zhang, Haocheng Xia, Yongjoo Park
University of Illinois Urbana-Champaign
sk227@illinois.edu, hengrui7@illinois.edu, hxia7@illinois.edu,
yongjoo@illinois.edu

Source Code: https://github.com/illinoisdata/mini-data-agent

Abstract

This document is both a working example and a usage guide for the arXiv paper template. It demonstrates how the template renders the title, authors, abstract, headings, citations, figures, tables, and mathematics, and it explains the document-class options and custom commands the template provides. To prepare your own paper, replace the title, author, abstract, and body text below with your own content while leaving the surrounding template machinery (`preprint.cls`, `preamble.tex`, and `command.tex`) intact. The abstract should be limited to a single paragraph; it is typeset in 10-point type inside the rounded title box at the top of the first page, and is optionally followed by a list of keywords.

Keywords: template, LaTeX, arXiv preprint, formatting instructions

1. Introduction

Data agents sit at the intersection of large-scale data exploration and complex reasoning-intensive tasks. However, real-world data environments are often far more heterogeneous and complex than the settings considered in existing studies, which largely focus on data stored in local files. We extend the evaluation setting along this spectrum of complexity: from purely local file storage, to cloud-based storage, and further to cloud data systems integrated with databases that support in-place computation.

2. Related Work (Haocheng)

TODO: Overview sentence

TODO: Change the citation style to numbered citations.

- Data agent: search and reasoning
- Related Datasets and Taxonomy

LLM Agents and Scaffolds LLM agents solve tasks by alternating between reasoning and acting in an environment, a pattern introduced by ReAct [28] and extended by Reflexion [19] and CodeAct [22]. In practice, this pattern is realized by agent scaffolds, harness code that mediates the model's access to an execution environment. Examples include SWE-agent [25], OpenHands [23], CrewAI [3], smolagents [18], and mini-SWE-agent [26], which impose varying constraints on how the model plans and executes tool calls. Applied to data tasks, such agents have been surveyed and classified by degree of independence [16]. However, how the data storage architecture itself limits or alters these reasoning loops remains unexamined. Our work fills this gap by evaluating a matrix of models, agents, and EQAbenchmarks across three data environments: local storage, S3 object storage, and distributed execution via Ray Data. Specifically, we profile how these data storage architectures alter agent execution pathways and quantify the resulting I/O costs at scale.

Benchmarks for Data Agents Benchmarks for data agents largely emphasize downstream execution. Works such as InfiAgent-DABench [7] and DSBench [9] evaluate statistical analysis and modeling, while DA-Code [8], ScienceAgentBench [2], DABstep [6], and DAComp [14] test code generation and multi-step workflows.

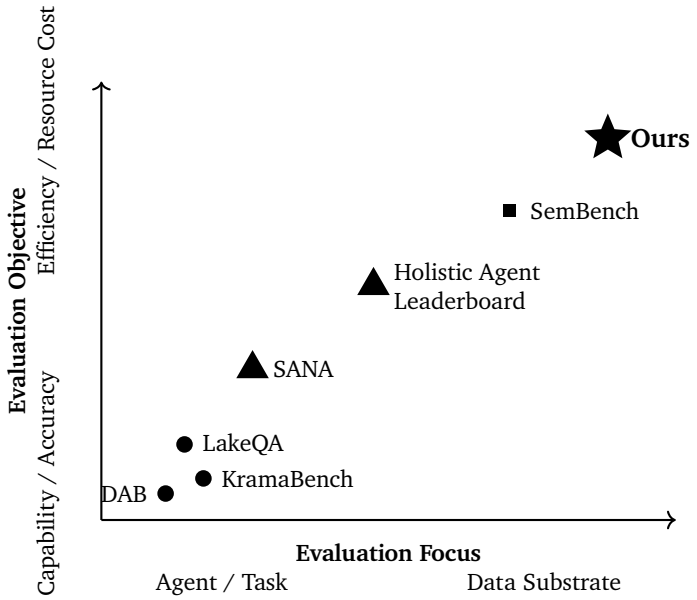


Figure 1: Conceptual landscape of data-agent evaluation. Prior work primarily studies agent capabilities or general execution costs, while our evaluation focuses on the efficiency implications of the underlying data substrate. (briefly, accuracy oriented-> efficiency oriented; agent centric -> data substrate centric)

However, these benchmarks all provide the required files upfront. By handing the agent its data directly, they leave its search capabilities unexamined.

To better reflect real-world data analysis, recent works increasingly evaluate search and task execution in tandem. Benchmarks such as CoDA-Bench [34], DataClawBench [32], and LakeQuest [20] test this by embedding target files among distractors. We evaluate three benchmarks from this class: KramaBench [12], LakeQA [21], and DAB [17]. Together, they cover diverse storage setups, including large data lakes, multi-source files, and relational database joins.

Computation Pushdown Reducing data movement by relocating computation toward data is a long-standing concern in data systems. Distributed execution frameworks exploit data locality [4, 30, 31], while near-data processing moves computation closer to storage [1, 5]. Disaggregated cloud storage has renewed interest in these ideas. PushdownDB [29] executes filters, projections, and aggregations within S3 to reduce data transfer and cost. Recent work further studies pushdown in cloud OLAP systems [27], near-data processing in cloud databases such as Taurus [15], and predicate pushdown in general-purpose pandas and Spark pipelines [33].

These studies show how computation placement impacts data movement in standard workloads. We extend this analysis to agentic QA, measuring how offloading computation to a remote cluster affects I/O costs during dynamic data analysis.

Cost and Diagnosis in Agent Evaluation Agent evaluation has moved beyond accuracy alone. Kapoor et al. [10] argue that reporting accuracy in isolation rewards needlessly expensive systems, and the Holistic Agent Leaderboard [11] provides standardized infrastructure for tracking cost alongside it. This multi-dimensional framework has since extended to data settings. SemBench [13] evaluates semantic query engines on quality, monetary cost, and latency jointly, mapping the distinct performance-versus-cost trade-offs across different architectures. SANA [24] takes a diagnostic approach, idealizing an agent's search, planning, and analysis components in turn to attribute end-to-end performance to each. In this paper, we focus on evaluating the agent based on the data I/O costs across different data storage architectures.

3. Methodology (Sky & Haocheng)

Here, we study the data movement created by an agent as it solves a variety of EQA tasks. Specifically, we measure the bytes that cross from storage to compute across different task styles, storage systems, and execution backends. As AI data agents are increasingly deployed to automate real-world analytics, overall efficiency is influenced not only by model reasoning but also by data access patterns. As a result, understanding the I/O costs associated with these access patterns is critical. Two agents can reach the exact same answer while moving vastly different amounts of data, depending on what they inspect and whether processing happens near the data.

[Yongjoo: state why we are studying that]

3.1. Benchmarks

[Yongjoo: we selected benchmarks that are purely about data analysis, among these we are filtering for the criteria we want]

We evaluate on three benchmarks for data agents. LakeQA is a multi-hop QA benchmark that requires agents to locate and combine evidence across a heterogeneous data lake. Due to cost constraints, we specifically use the LakeQA-mini suite, which spans 135 tasks over a 14.3GB data lake. KramaBench evaluates multi-step data-science QA across 104 tasks. Agents must locate relevant data across diverse formats and perform the necessary cleaning, transformation, and analysis to compute the answer. Lastly, DAB contains 54 tasks that test multi-database querying under realistic schema noise, requiring agents to locate relevant tables within large schemas and then resolve fuzzy joins and text extraction across separate database systems.

We select these benchmarks because each couples **data search** with task execution, reflecting real-world workflows where data must be located before analysis. Collectively, they span a diverse range of data formats, from plain text and tabular exports to scientific encodings and relational databases, reflecting the heterogeneity of real-world data. In addition, each benchmark supports deployment across all three of our storage environments (local disk, cloud storage, and cloud storage with co-located compute). Finally, all tasks can be completed through standard bash interfaces, which enables direct execution by agents such as mini-SWE-agent.

3.2. System Architecture

This section describes the system architecture used in our study. Our aim is to measure how the I/O costs of agentic data analysis are shaped by the underlying storage and execution infrastructure. To do this, we evaluated three configurations that reflect common industry deployments: local storage, cloud storage, and cloud storage with co-located compute. These configurations differ in where the data resides and where computation over the data can occur, which determines how many bytes must cross the boundary between storage and the agent. To isolate the effect of the storage and execution infrastructure itself, we use the same agent, mini-SWE-agent, across all three configurations. The remainder of this section describes each configuration in turn.

3.2.1. Local Storage

The local architecture serves as our baseline. Here, the data and agent are co-located on a single machine, and the data are stored as ordinary disk files. This allows the agent to read the data directly via the filesystem. Because the agent and data reside together, no bytes cross the network.

3.2.2. Cloud Storage

Modern cloud analytics relies heavily on cloud storage systems such as Amazon S3, which physically separate compute engines from storage infrastructure [**?**]. We mirror this architecture using MinIO, an S3-compatible object storage server deployed across four storage nodes external to the agent environment. Without filesystem access, the agent must list and retrieve objects through the S3 API before any analysis can begin. This allows us to measure the network latency and data transfer costs incurred when compute and storage are disaggregated.

3.2.3. Cloud Storage with Co-Located Compute

Co-locating compute with cloud storage allows computation to move to data rather than data to computation. We co-locate the four-node MinIO deployment with a Ray cluster, running a compute worker on each storage node. This gives the agent two ways to reach the data: retrieve objects directly from S3 as in Section 3.2.2, or ship code to the cluster and compute in place, returning only the final result. Neither execution path is privileged,

Axis	Question	Primary metric	Status
Premise	Does the substrate change the answer?	judged accuracy	preliminary eviden
I. Cost	What does it change instead?	tokens, wall time, bytes	partial coverage
II. Placement	Does the agent use the pushdown door?	`uses_ray`, pushdown	preliminary eviden
III. Reproducibility	Is the cost of a run reproducible?	CV over replicates	preliminary eviden
IV. [Haocheng: Insight for agent optimization]	ideal case: data movement cost (prefetch)	occupation	

Table 1: Research-question map. Each axis is developed in its own subsection below.

allowing us to evaluate whether the agent effectively leverages cluster computing. We measure changes in I/O costs between direct remote storage access (with local computation) and cluster-side execution.

These *approaches* go inside the subsection 2: system architecture

A1. Agent + pure local storage
A2. Agent + cloud storage
A3. Agent + cloud db & storage

The opening of this section should state our intention: what we aim to study? Goal

These are subsection: (1) benchmarks: why these are good representatives?, (2) system architecture (agent host & cluster configuration), (3) metrics and measurements: why are we measuring them? we do they matter?

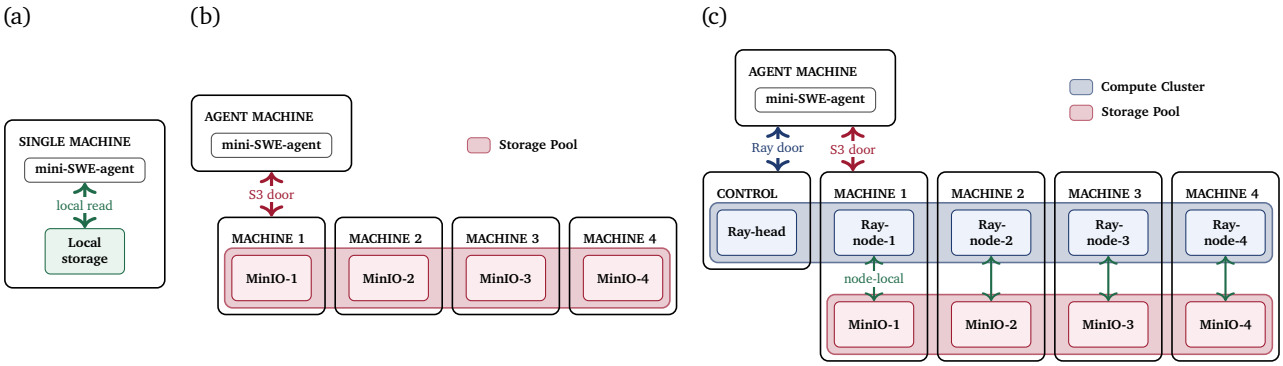


Figure 2

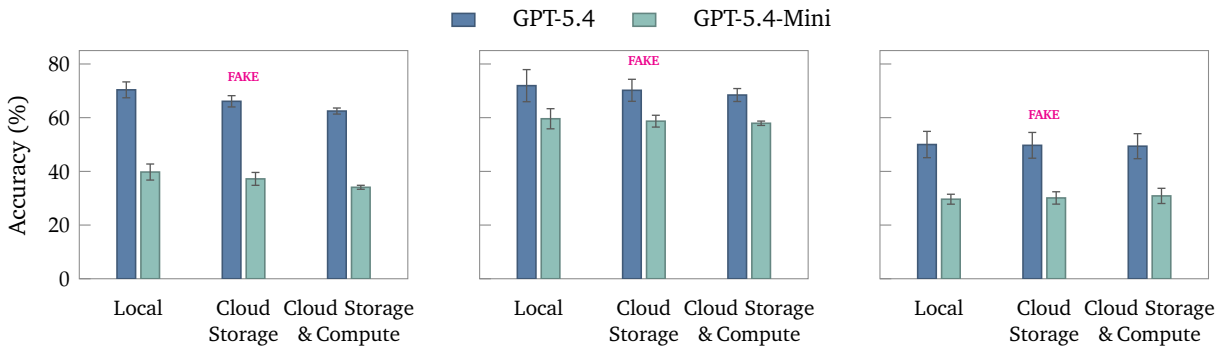
4. Analysis & Findings

Our three architectures (*A1* local, *A2* object storage, *A3* distributed cluster) hold the agent scaffold, the model, the prompt, and the task set fixed, and vary only where the bytes live and how the agent may reach them. This design lets us separate a question about *capability* from a question about *cost*: does moving the data change what the agent answers, or only what the answer costs to produce? We organize the analysis around one premise and four research axes, summarized in Table 1. The premise (§4.1) establishes that accuracy is substrate-invariant, which is what licenses the remaining axes to treat cost, not correctness, as the dependent variable.

(Label check: §4 refers to the three arms as Local / S3 / Ray-Data. Once §3.2.3 is written, align these with the *A1–A3* labels used in the methodology.)

4.1. Premise: Accuracy Is Substrate-Invariant

RQ0. *Does relocating the corpus from local disk to object storage, or to a distributed cluster with a pushdown door, change the accuracy of the agent's final answer?* Quick answer might be no.



(a) LakeQA (b) KramaBench (c) DAB

Figure 3: Accuracy by storage backend across three benchmarks. Error bars show ± one standard deviation across runs.

Why this matters. A null result on accuracy is the enabling condition for the rest of the paper. It means the substrate is not a confound on task difficulty, so any difference we observe downstream is a difference in *how* the same answer was reached. It also sharpens the argument of Kapoor et al. [10]: two systems that are indistinguishable on the leaderboard here differ substantially on every cost dimension we measure.

4.2. Axis I: What the Substrate Changes Instead — the Cost Profile

RQ1.1 (Token inflation). *How much more model input does a remote substrate consume for the same task set at the same accuracy?*

RQ1.2 (Locus of inflation). *Is the inflation driven by more agent steps, or by longer context per step?* This is the sharpened form of the original "correlation between data storage and # steps per trajectory" question. Table **??** suggests the latter: step counts move by well under one step across substrates while input tokens move by tens of percent, implicating longer per-step observations (directory listings, transfer chatter, retry output) rather than longer trajectories. Testable directly from the per-command `out_KB` column against `llm_calls`.

RQ1.3 (Latency decomposition). *Where does wall-clock time go — model latency, in-container command execution, or in-cluster execution?* Our instrumentation records `llm_ms` and `wall_ms` disjointly per command and `ray_job_ms` per submitted job, which decomposes a run into seven bands. Preliminary inspection of KramaBench Ray-Data traces shows model latency dominating command latency by roughly 3–5× per task, meaning the substrate's byte cost is largely hidden behind the model's own think time at this corpus scale.

RQ1.4 (The local anomaly). *Why is the Local arm the* slowest *on LakeQA?*

RQ4.1 (Search precision). *What is the touched/required ratio, and does the substrate change it?* If discovery dominates LakeQA as claimed, this ratio should be large there and near unity on KramaBench.

RQ4.2 (Waste decomposition). *Of the bytes that were not strictly required, how many were spent looking in the wrong place versus reading too much of the right place?* Per-process byte attribution separates exploratory listing and sniffing from bulk reads of the correct object.

RQ4.3 (Empirical support for §3.1). *Do the three benchmarks in fact separate on this axis?* Plotting all three on a common touched/required axis converts §3.1 from an assertion into a figure, and is the cleanest empirical justification for using three benchmarks rather than one.

4.3. Axis II: The Pushdown Door the Agent Declines to Use

The Ray-Data arm is the only setting in which the agent has a genuine placement choice: it may pull bytes to itself through the S3 door, or ship code to the data through the Ray door. Which door it walks through is not prescribed by the prompt — both are described neutrally — so the choice is itself a measurement.

RQ2.1 (Adoption). *How often does the agent choose the pushdown door when it is freely available?* Rarely.

RQ2.2 (Selectivity). *Under what conditions* does *the agent push down?* Bucketing tasks by logical bytes read tests whether the agent's choice is at least data-size-sensitive — i.e. whether it reserves the Ray door for the tasks where it pays.

RQ2.3 (Payoff). *When the door is used, does it actually reduce cost, and does it change accuracy?* The nine independent Ray-Data replicates of KramaBench give a natural paired design: the same task is solved via the Ray door in some runs and via the S3 door in others, which controls for task difficulty without needing a difficulty model. Both the per-task byte columns and the per-task judged score are keyed identically and can be joined directly.

RQ2.4 (Diagnosis, and the DAG question).This is optional *Why does the agent decline?* Trajectory inspection distinguishes three failure modes: the agent never considers the door; it attempts a submission, fails, and falls back; or it finds the submit-and-poll protocol too costly in steps relative to a direct read. This is the natural home for the observation that motivated our earlier question: in Ray-Data traces the agent typically *inspects* the data and then writes one full query, rather than building intermediate checkpoints — even without being instructed to plan. *If a LakeQA task admits a complete operator DAG generated up front, is it still necessary to keep the LLM inside the step-by-step execution loop?* A "typed IR + schema checker" scaffold is one candidate answer, and this axis supplies the evidence for whether it would pay.

4.4. Axis III: Is the Cost of a Run Reproducible?

RQ3.1 (Magnitude). *How much does the cost of an agentic run vary across identically configured repetitions, and how does that compare to the variance in accuracy?* Across nine independent KramaBench Ray-Data runs with identical configuration, judged accuracy has CV $= 2.3\%$ while logical bytes read has CV $= 55.6\%$, a $24\times$ gap. The largest run reads $5.7\times$ the bytes of the smallest while scoring within 0.05 of it. **Accuracy is reproducible; cost is not.**

RQ3.2 (Concentration). *Is the variance carried by a few volatile tasks or spread across the whole task set?* Per-task coefficients of variation and a Lorenz curve over per-task bytes distinguish a heavy-tail regime (a handful of tasks occasionally trigger a full-corpus scan) from a diffuse one.

RQ3.3 (Mechanism). *Does tool-chain divergence explain the byte variance?* Representing each run's per-task behavior as a vector over the tool combinations in Table **??** lets us test whether runs that read more bytes are the runs that chose a different access pattern — the "different tool chains used by different runs" hypothesis, made falsifiable.

RQ3.4 (Methodological consequence). *Is a single-run I/O number meaningful?* On this evidence, no: a single run's byte figure carries a $\pm 55\%$ error bar that no reader can see. We propose reporting agentic I/O as a mean over ≥ 3 replicates with an explicit spread, and we treat this as a contribution in its own right — prior cost-aware agent evaluations report accuracy with variance but cost without it.

4.5. Secondary Axes

Agent working-memory budget. An existing but unlabeled sweep varies a per-run resource budget across S3 runs of LakeQA and KramaBench. On LakeQA the larger setting scores 0.752 ± 0.021 against 0.702 ± 0.024 at the baseline, which would be the only configuration knob we have found that moves accuracy at all. **(Sky/Haocheng:**

confirm what the `100mb` / `1gb` run labels actually varied (agent container `-memory`? DuckDB `memory_limit`?) before we write this up — the run artifacts do not record it.)

Model scale. We currently have two models (`gpt-5.4` and `gpt-5.4-mini`), and the mini runs are too few per cell to support a claim (n=2 with SD = 0.30 in the worst cell). Establishing whether substrate effects interact with model capability requires filling this out.

4.6. Measurement Coverage and Threats to Validity

Coverage gap. The modern byte-accounting columns (per-process logical disk reads, object-store receive, database receive) are complete for DAB on all three substrates, but the `gpt-5.4` Local and S3 runs of LakeQA and KramaBench predate them and retain only aggregate reports. Axis I's cross-substrate byte comparison is therefore currently defensible only on DAB; closing it requires re-running Local and S3 under instrumentation ($\geq$ 3 replicates each).

Comparability windows. The harness changed several times during data collection in ways that alter the agent's capability rather than merely its instrumentation: a CPU quota was imposed on the agent sandbox, the cache-hit metric changed definition, the container image gained additional CLI tools and libraries, and the agent container became persistent-with-reset rather than freshly created per task. Runs on either side of these boundaries are not byte-comparable. Before pooling, we construct a run $\times$ validity-window matrix and report only within-window aggregates.

Instrumentation neutrality. Byte counters are read from the containers' own cgroup and network counters plus an `LD_PRELOAD` read shim; the agent's data path (network placement, endpoints, credentials) is applied identically whether or not measurement is enabled, so a measured run and an unmeasured run are the same experiment.

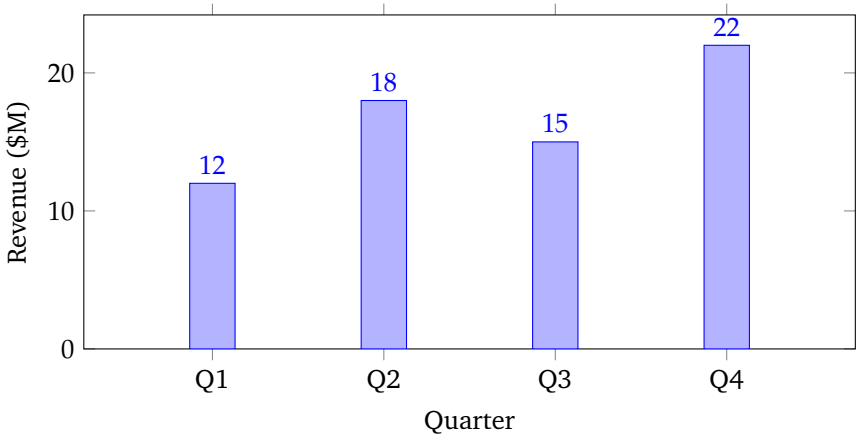


Figure 4: Quarterly revenue for fiscal year 2025, showing steady growth with a dip in Q3.

5. Conclusion

[Haocheng: load occupied a large proportion of the total time cost, whether can we analyze the possible benefits from prefetching? it is possible to hide the latency for RayData backend on some benchmarks]

References

[1] Rajeev Balasubramonian, Jichuan Chang, Troy Manning, Jaime H. Moreno, Richard Murphy, Ravi Nair, and Steven Swanson. Near-data processing: Insights from a micro-46 workshop. *IEEE Micro*, 34(4):36–42, 2014. doi: 10.1109/MM.2014.64.

[2] Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations (ICLR)*, 2025.

[3] CrewAI. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. https://github.com/crewAIInc/crewAI, 2024. Accessed: 2026-08-13.

[4] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, pages 137–150, San Francisco, CA, 2004. USENIX Association.

[5] Jaeyoung Do, Yang-Suk Kee, Jignesh M. Patel, Chanik Park, Kwanghyun Park, and David J. DeWitt. Query processing on smart ssds: Opportunities and challenges. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 1221–1230, 2013. doi: 10.1145/2463676.2465295.

[6] Alex Egg, Martin Iglesias Goyanes, Friso Kingma, Andreu Mora, Leandro von Werra, and Thomas Wolf. DABstep: Data agent benchmark for multi-step reasoning. *arXiv preprint arXiv:2506.23719*, 2025.

[7] Xueyu Hu, Ziyu Zhao, Shuang Wei, Ziwei Chai, Qianli Ma, Guoyin Wang, Xuwu Wang, Jing Su, Jingjing Xu, Ming Zhu, Yao Cheng, Jianbo Yuan, Jiwei Li, Kun Kuang, Yang Yang, Hongxia Yang, and Fei Wu. InfiAgent-DABench: Evaluating agents on data analysis tasks. In *International Conference on Machine Learning (ICML)*, 2024.

[8] Yiming Huang, Jianwen Luo, Yan Yu, Yitong Zhang, Fangyu Lei, Yifan Wei, Shizhu He, Lifu Huang, Xiao Liu, Jun Zhao, and Kang Liu. DA-Code: Agent data science code generation benchmark for large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 13487–13521, 2024.

[9] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. DSBench: How far are data science agents from becoming data science experts? In *International Conference on Learning Representations (ICLR)*, 2025.

[10] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=Zy4uFzMviZ.

[11] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=vUaY1t64ZZ.

[12] Eugenie Lai, Gerardo Vitagliano, Ziyu Zhang, Om Chabra, Sivaprasad Sudhir, Anna Zeng, Anton A. Zabreyko, Chenning Li, Ferdi Kossmann, Jialin Ding, J. Chen, M. Markakis, M. Russo, W. Wang, Z. Wu, M. Cafarella, L. Cao, S. Madden, and T. Kraska. KramaBench: A benchmark for AI systems on data-to-insight pipelines over data lakes. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2506.06541; OpenReview fZf UdeCC5X.

[13] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. SemBench: A benchmark for semantic query processing engines. *arXiv preprint arXiv:2511.01716*, 2025.

[14] Fangyu Lei, Jinxiang Meng, Yiming Huang, Junjie Zhao, Yitong Zhang, Jianwen Luo, Xin Zou, Ruiyi Yang, Wenbo Shi, Yan Gao, Shizhu He, Zuo Wang, Qian Liu, Yang Wang, Ke Wang, Jun Zhao, and Kang Liu.

DAComp: Benchmarking data agents across the full data intelligence lifecycle. In *International Conference on Learning Representations (ICLR)*, 2026. OpenReview EtzJy9yI5J.

[15] Shu Lin, Arunprasad P. Marathe, Per-Åke Larson, Chong Chen, Calvin Sun, Paul Lee, and Weidong Yu. Near data processing in taurus database. *arXiv preprint arXiv:2506.20010*, 2025.

[16] Yuyu Luo, Guoliang Li, Ju Fan, and Nan Tang. Data agents: Levels, state of the art, and open problems. In *Companion of the International Conference on Management of Data (SIGMOD Companion)*, pages 571–579. ACM, 2026.

[17] Ruiying Ma, Shreya Shankar, Ruiqi Glen Chen, Yiming Lin, Sepanta Zeighami, Rajoshi Ghosh, Abhinav Gupta, Anushrut Gupta, Tanmai Gopal, and Aditya G. Parameswaran. Can AI agents answer your data questions? a benchmark for data agents. *arXiv preprint arXiv:2603.20576*, 2026.

[18] Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. smolagents: A barebones library for agents. https://github.com/huggingface/smolagents, 2025. Accessed: 2026-08-13.

[19] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[20] Michael Solodko, Steven Gong, Guangwei Yu, Satya Krishna Gorti, Jesse C. Cresswell, and Victor Zhong. LakeQuest: Benchmarking ai data agents on natural language discovery and insight generation over data lakes. *arXiv preprint arXiv:2607.12310*, 2026.

[21] Haonan Wang, Jiaxiang Liu, Yurong Liu, Austin Senna Wijaya, Tianle Zhou, Eden Wu, Yijia Chen, Wanting You, Reya Vir, Daniela Pinto, Grace Fan, Yusen Zhang, Juliana Freire, and Eugene Wu. LakeQA: Exploratory question answering over a million-scale data lake. In *International Conference on Machine Learning (ICML)*, 2026.

[22] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024.

[23] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[24] Austin Senna Wijaya, Jiaxiang Liu, Haonan Wang, and Eugene Wu. SANA: What matters for QA agents over massive data lakes? *arXiv preprint arXiv:2606.13904*, 2026.

[25] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[26] John Yang, Carlos E. Jimenez, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. mini-swe-agent. https://github.com/SWE-agent/mini-swe-agent, 2025. Accessed: 2026-08-13.

[27] Yifei Yang, Xiangyao Yu, Marco Serafini, Ashraf Aboulnaga, and Michael Stonebraker. Enhancing computation pushdown for cloud olap databases. *arXiv preprint arXiv:2312.15405*, 2023.

[28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[29] Xiangyao Yu, Matt Youill, Matthew Woicik, Abdurrahman Ghanem, Marco Serafini, Ashraf Aboulnaga, and Michael Stonebraker. Pushdowndb: Accelerating a dbms using s3 computation. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, 2020. doi: 10.1109/ICDE48307.2020.00174.

[30] Matei Zaharia, Dhruba Borthakur, Joydeep Sen Sarma, Khaled Elmeleegy, Scott Shenker, and Ion Stoica. Delay scheduling: A simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European Conference on Computer Systems (EuroSys)*, pages 265–278, 2010. doi: 10.1145/1755913.1755940.

[31] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 15–28. USENIX Association, 2012.

[32] Qiaohong Zhang, Weihao Ye, Jialong Chen, Yi Luo, BoYuan Li, Bowen Deng, Zibin Zheng, Jianhao Lin, Wei-Shi Zheng, and Chuan Chen. DataClawBench: An agent benchmark for exploratory real-world financial data analysis. *arXiv preprint arXiv:2605.02503*, 2026.

[33] Robert Zhang, Eric Hayden Campbell, Dixin Tang, and Işil Dillig. Optimal predicate pushdown synthesis. *Proceedings of the ACM on Programming Languages*, 10(PLDI):46, 2026. doi: 10.1145/3808312.

[34] Yuxin Zhang, Ju Fan, Meihao Fan, Shaolei Zhang, and Xiaoyong Du. CoDA-Bench: Can code agents handle data-intensive tasks? In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2606.15300.